TU WIEN Informatics

Towards Interoperability between Agentic AI Frameworks through Semantic Representation

DIPLOMA THESIS

submitted in partial fulfillment of the requirements for the degree of

Diplom-Ingenieur

in

Business Informatics

by

Dani Lippmann BSc
Registration Number 12332231

to the Faculty of Informatics

at the TU Wien

Advisor: Forename Surname
Assistance:

Vienna, June 30, 2026

Dani Lippmann BSc	Forename Surname

Declaration of Authorship

Dani Lippmann BSc

I hereby declare that I have written this thesis independently, that I have completely specified the utilized sources and resources and that I have definitely marked all parts of the work – including tables, maps and figures – which belong to other works or to the internet, literally or extracted, by referencing the source as borrowed.
I further declare that I have used generative AI tools only as an aid, and that my own intellectual and creative efforts predominate in this work. In the appendix "Overview of Generative AI Tools Used" I have listed all generative AI tools that were used in the creation of this work, and indicated where in the work they were used. If whole passages of text were used without substantial changes, I have indicated the input (prompts) I formulated and the IT application used with its product name and version number/date.

Vienna, June 30, 2026

Dani Lippmann BSc

Acknowledgements

Abstract

Contents

Abstract **vii**

Contents **ix**

1 Introduction **1**

2 Related Work **3**
2.1 Semantic Representations for Agentic and AI Systems 3
2.2 Multi-agent Systems and Frameworks 4
2.3 Model transformation and cross-framework interoperability 5
2.4 Research Gap 5

3 Research Methods **7**
3.1 Requirement Elicitation through Literature Review and Framework Analysis 7
3.2 Ontology Engineering using Ontology Development 101 9
3.3 Bidirectional Transformation using the OSCIN Method 9
3.4 Evaluation 10

4 Agentic AI Concepts and Framework-level Implementation **13**
4.1 Agent Definition 15
4.2 Perception Layer and Task Definition 17
4.3 Coordination Mechanism 19
4.4 Reasoning Engine 22
4.5 Memory Architecture 24
4.6 Tool Integration 26
4.7 Guardrails and Governance Mechanisms 28
4.8 Consolidated Requirements 30

5 A Framework-Agnostic Semantic Representation **35**
5.1 Scope and Competency Questions 35
5.2 Reuse Evaluation 36
5.3 Defining Classes and Properties 37

6 Bidirectional Transformation Method **43**
6.1 OSCIN Instantiation 43

7 Evaluation **49**
7.1 Evaluation Design 49
7.2 Extraction Evaluation 50
7.3 Generation Evaluation 50
7.4 Round-Trip and Cross-Framework Evaluation 50
7.5 Comparison with LLM-Based Baseline 50
7.6 Discussion of Results 50

8 Discussion **53**
8.1 Interpretation of Findings 53
8.2 Threats to Validity 53
8.3 Relation to Research Questions 53

9 Conclusion **55**
9.1 Summary of Contributions 55
9.2 Limitations and Future Work 55

Overview of Generative AI Tools Used **57**

List of Figures **59**

List of Tables **61**

List of Algorithms **63**

Bibliography **65**

CHAPTER 1

Introduction

Over the past several years, agentic AI systems have moved from research prototypes to real world production deployments, driven by organizational demand to automate complex tasks without continuous human oversight [BKN+25]. In contrast to conventional AI systems, agentic systems are capable of planning, delegating and acting autonomously across extended task [AKA25]. However, as these systems scale to handle more sophisticated and distributed workflows, their underlying architectures become increasingly complex, exposing severe challenges in interoperability, scalability as well as system integration [DBM25].

This is particularly critical as agentic AI systems, in practice, are frequently developed in a monolithic and ad hoc manner, relying on framework-specific abstractions and tightly coupled implementations [SRK26, EKEK26]. Frameworks such as LangGraph, CrewAI, and AutoGen each realize similar agentic concepts through fundamentally different code-level constructs (e.g. graph-based state machines, role-based agent teams, and conversation-driven orchestration). Architectural decisions and coordination logic are often hard-coded into source code, thereby limiting modularity and transferability across systems [EKEK26]. This lack of standardized and transferable architectural semantics has been identified as a central challenge for agentic AI, as it hinders the comparison of implementations, the replication of results, and generalization across domains [SRK26].

Existing semantic representations partially address these challenges by providing expressive conceptual vocabularies for agents, tasks and workflows. More recent work such as AgentO [EKEK26] additionally demonstrates the feasibility of extracting agentic workflows from framework-specific source code into a unified ontology. However, these approaches remain focused on unidirectional extraction and do not model constructs such as conditional control flow, state management, or prompt-level semantics at sufficient depth to support bidirectional transformation between frameworks. This unidirectional limitation shows that existing representations are not expressive enough to serve as a shared

intermediate layer between frameworks and therefore cannot support the systematic migration or comparison of agentic architectures across different implementations.

This thesis aims to contribute to addressing the challenges above and enrich the discourse on the topic of Agentic AI Systems and their semantic presentation. Specifically, we aim to answer the main research question:

> *To what extent can semantic representations enable interoperability between agentic AI frameworks?*

Answering this question requires demonstrating that the vocabulary is rich enough to carry the architectural information of real agentic systems in both directions. This means from framework-specific implementations into a formal representation, and from that representation back into executable code. The thesis approaches the question through round-trip transformation, using the fidelity of extraction and regeneration as a measurable indicator of how far a shared ontology can close the gap between frameworks. To structure this research, the main question is split into four sub-questions:

- **RQ1:** Which architectural constructs must a semantic representation support to capture agentic AI systems implemented in CrewAI, AutoGen and LangGraph?
- **RQ2:** To what extent can semantic representations of agentic AI systems be systematically derived from their source code?
- **RQ3:** To what extent can the source code of agentic AI systems be generated from their semantic representations?
- **RQ4:** Which methods can be used to adequately evaluate the quality of extraction and generation of agentic AI systems from framework-specific source code?

RQ1 establishes the conceptual foundation by identifying what a shared representation must express to capture the constructs found in contemporary agentic AI frameworks. The outcome of this phase is a list of requirements a semantic representation needs to fulfill, to function as a framework-agnostic semantic representation. This list also acts as the guideline of the subsequent ontology evaluation, where we will check whether our ontology provides a higher concepts coverage than existing data models.

RQ2 and RQ3 form the two halves of the round-trip: the first concerns lifting framework-specific implementations into the semantic representation, the second concerns regenerating executable implementations from it. RQ4 addresses how the results of both directions can be evaluated rigorously rather than asserted, and grounds the evaluation through comparison against a large language model-driven baseline.

CHAPTER 2

Related Work

This chapter reviews prior work relevant to the thesis in three areas: semantic representations of agentic and AI systems, the landscape of contemporary multi-agent frameworks, and model transformation approaches that bridge source code and semantic models.

2.1 Semantic Representations for Agentic and AI Systems

Alongside the development of agentic AI frameworks, several semantic representations have been proposed to capture the structure and interactions of complex AI systems. The BEAM ontology provides a high-level semantic model for representing AI systems within their operational and organizational context [EPK25]. BEAM is an extension of the orginial Boxology [HT19] and focuses on systems, resources and contextual elements, enabling structured documentation of AI-based solutions and their deployment environments. While BEAM supports architectural description and contextual reasoning, it remains largely agnostic to fine-grained agentic control structures and execution semantics, which limits its suitability for detailed modeling of multi-agent workflows.

AgentO is a recent ontology designed specifically for agentic AI systems and is the primary semantic foundation on which the present work builds [EKEK26]. Its central premise is that contemporary agentic frameworks lack a shared conceptual model: each framework encodes its logic directly in code, resulting in monolithic implementations that hinder reusability, interoperability and auditing across systems. AgentO addresses this by offering a standardized RDF/OWL vocabulary for the recurring concepts of the agentic paradigm (agents, goals, tasks, tools, resources, workflows and their interrelations) so that agentic systems can be represented declaratively and independently of any particular framework.[EKEK26] also developed an LLM-driven extraction pipeline that transforms source code from four agentic frameworks (AutoGen, CrewAI, LangGraph and Mastra AI) into instances of the ontology, producing a knowledge graph of 66 agentic patterns. The authors demonstrate the utility of this representation through three use cases: declarative

reconstruction of agentic workflows, reuse of agent and task configurations across problem contexts, and auditing of agentic implementations for transparency and traceability [EKEK26].

2.2 Multi-agent Systems and Frameworks

Multi-agent systems are not a new research field. Work on coordinated autonomous agents dates back decades [WJ95, Woo09]. Classical MAS research was driven by questions of distributed problem solving, concurrent rationality and open systems. What large language models have changed is not the underlying paradigm but the foucs. Agents are now natural-language reasoners with broad general competence rather than narrow symbolic planners, and the coordination problem has accordingly shifted from symbolic message exchange to prompt engineering, tool orchestration and workflow control [Bot25, GCW+24]. The transition towards LLM-based multi-agent systems was motivated in large part by tasks that exceed the context window, domain coverage or reasoning budget of any single model. Early LLM-based frameworks explored different communication topologies to manage this coordination, broadly categorizing interactions into chain (waterfall), star (hub-and-spoke), and mesh (swarm) patterns[VRB26]. Among these three agentic AI frameworks have become particularly influential in both research and production settings: AutoGen, CrewAI and LangGraph [Bot25]. Each realizes multi-agent collaboration through a different architectural commitment.

AutoGen treats multi-agent collaboration as a conversation between conversable entities [WBZ+23]. Its underlying model is dialogical: agents, tools and humans are all first-class participants in a shared message stream, and the system's behavior emerges from the rules that govern who speaks next. CrewAI treats multi-agent collaboration as role-based teamwork. Agents are defined by personas and responsibilities, organized into crews, and coordinated through processes that resemble organizational structures[Cre25]. LangGraph, developed by LangChain, treats multi-agent collaboration as a programmable workflow. Its underlying model is computational: the system is a state machine whose transitions are explicit and whose execution is traceable. Agent identity is secondary to graph structure, which places LangGraph closer to workflow engines and flow-based programming than to either dialogue systems or role-based teams [Lan25d, Cla26].

These three framings (dialogue, role-based team and programmable workflow) represent genuinely different theories of what coordinated LLM agents are. They agree on the vocabulary of the domain while disagreeing on which concept is primary. This divergence is what motivates a shared ontological representation, and it is the starting point of the present work.

2.3 Model transformation and cross-framework interoperability

In the context of agentic AI systems, recent work such as AgentO demonstrates the feasibility of extracting agentic workflows from framework specific implementations into a unified semantic representation using Large Language Models [EKEK26]. These approaches enable declarative reconstruction, cross framework comparison, and workflow auditing. However, they remain focused on one directional extraction and analysis and do not provide mechanisms for regenerating framework specific implementations from semantic models.

Model Driven Engineering (MDE) provides an established foundation for closing this gap. In MDE, models are treated as primary development artefacts from which executable code is generated rather than as mere documentation [SRF18, FC18]. The paradigm has been applied to many domains but not, so far, to agentic AI frameworks. A concrete instantiation relevant to the present work is the Ontology-based Source Code Interoperability (OSCIN) method [dAZS21]. OSCIN separates syntactic from semantic abstraction. Source code is first parsed into a syntactic structure specific to its host language, and then mapped to a shared ontology representing a common subdomain. By routing heterogeneous source languages through a central operational ontology, this method mitigates both syntactic and semantic heterogeneity and exposes a single abstraction layer to downstream tools.

However, similar to extraction method developed by AgentO[EKEK26], OSCIN focuses primarily on interoperability and perspective-based analysis rather than the feasiblity of bidirectional transformation. While it successfully abstracts source code into rich, queryable semantic graphs that can unify different languages under a common domain consensus, it serves as an analytical framework and does not natively provide mechanisms to dynamically regenerate executable, framework-specific implementations from a semantic representation.

2.4 Research Gap

The research gap this thesis addresses is whether a shared ontology can represent agentic AI systems with enough details that framework-specific implementations can be extracted into the ontology and regenerated from it, and how much semantic information is lost in each direction. Existing work on AgentO has shown that extraction alone is possible, but the inverse direction has not been attempted, and without it the adequacy of the representation cannot be rigorously tested. Round-trip transformation therefore serves as both the method and the measurement. It forces the ontology to carry enough structure to reconstruct executable systems, and the fidelity loss it exposes becomes a direct indicator of where the shared representation succeeds and where it reaches its limits.

CHAPTER 3

Research Methods

This research follows the Design Science Research (DSR) approach as defined by [HMPR04] and [Hev07]. Requirement elicitation addresses the *relevance cycle*, ontology engineering bridges the *rigor* and *design cycles*, and the bidirectional transformation method together with its evaluation completes the *design cycle*.

3.1 Requirement Elicitation through Literature Review and Framework Analysis

The requirement elicitation phase aims to identify representational requirements for a semantic model of agentic AI systems that supports bidirectional transformation. The phase is structured in two steps.

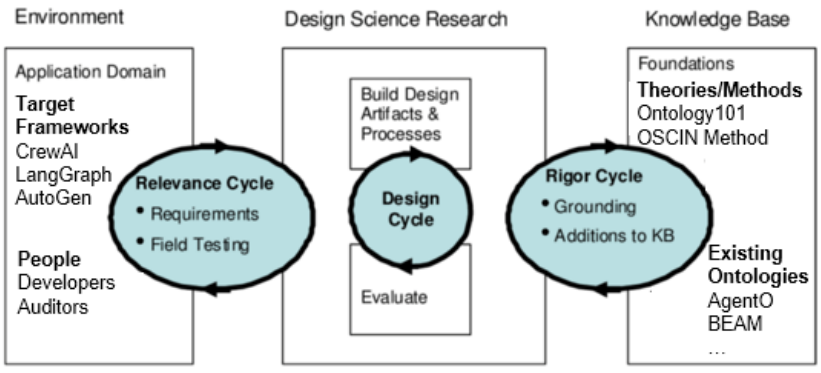

Figure 3.1: Three cycle view of DSR according to [Hev07]

The first step is a literature review conducted according to [GB09]. The review follows a structured process of identifying relevant sources, extracting recurring architectural concepts and organizing them into a cross-paper concept matrix (see Chapter 4). The search string ("agentic AI" OR "multi-agent systems") AND ("ontology" OR "representation" OR "architecture") was applied to Google Scholar and Semantic Scholar with a cut-off date of April 20, 2026.

Because the research field is new and rapidly evolving, preprints were included alongside peer-reviewed publications. The initial result set was extended through backward snowballing: references cited in the papers retained after full-text screening were examined and included when they satisfied the same selection criteria. The results were screened in three stages: title, abstract and full text. A paper was included if it (i) addressed the architectural design of LLM-based or classical multi-agent systems, (ii) was written in English and (iii) was accessible in full text. A paper was excluded if it (i) focused on single-agent systems without multi-agent considerations, (ii) treated multi-agent systems purely as an application case study rather than analyzing their design, (iii) concentrated on agent communication protocols or runtime behavior rather than on architectural structure. Classical multi-agent systems references such as Wooldridge's foundational work [WJ95, Woo09] were included selectively as conceptual anchor points for the taxonomy.

The retained papers were then analyzed to extract the architectural concepts each paper identifies as constitutive of agentic AI systems. Concepts were coded individually per paper and subsequently aggregated across the corpus. Semantically related concepts were grouped into higher-level categories, which form the main concepts of the taxonomy. Finer-grained concepts that refine or specialize a main concept are retained as sub concepts beneath it. The result is a two-tier concept matrix that records, for each paper in the corpus, which main concepts and sub concepts it addresses.

The second step is an analysis of framework-specific artefacts. The main-concepts identified in the literature review are examined at the source code level across three agentic AI frameworks that represent fundamentally different architectural paradigms: sequential role-based (CrewAI), graph-based (LangGraph) and conversation-driven (AutoGen). For each concept, the analysis documents how, and whether, the concept is realized in each framework. Developer documentation [Cre25, Mic25, Lan25d] and selected open-source code bases of the three frameworks are the primary sources. The specific versions analyzed are CrewAI 1.13.0, AutoGen (`autogen-agentchat`) 0.7.5, and LangGraph at the state of its main branch as of April 20, 2026. The output of this step is a set of requirements that a framework-agnostic ontology must fulfill in order to represent agentic AI systems across the three target frameworks. Each requirement is traceable back to at least one framework-level construct, one literature concept or both, ensuring that the requirement set is grounded.

3.2 Ontology Engineering using Ontology Development 101

The second methodological step is the development of a formal semantic representation for agentic AI systems. This thesis proposes the development of an framework-agnostic ontology to semantically represent the requirements of phase one. Ontology engineering is conducted following the Ontology Development 101 methodology [NM01] and applied in four iterative activities.

First, the scope and domain of the ontology are determined based on the concepts and requirements list produced in the previous phase. The scope is bounded by the three target frameworks and the agentic AI concepts identified across them. Second, competency questions are formulated to define what the ontology must be able to answer. These questions are derived directly from the requirements elicitation and serve both as design guidance during modeling and as a validation criterion during evaluation. Third, existing vocabularies are evaluated for reuse. AgentO [EKEK26] is adopted as the primary starting point because it already covers the recurring concepts of the agentic paradigm and builds on established upper vocabularies (PROV-O, P-Plan and BEAM). Concepts that are not expressible in AgentO, as identified through the requirement analysis, are introduced as extensions rather than as a parallel vocabulary. This includes constructs required for bidirectional transformation. Fourth, the extended ontology is formalized in OWL to ensure machine interpretability and compatibility with existing Semantic Web standards. The resulting ontology is referred to as AgentOscin.

3.3 Bidirectional Transformation using the OSCIN Method

The third methodological step is the construction of a bidirectional transformation method that links framework-specific source code of agentic AI systems to AgentOscin. The method follows the five phases of the OSCIN method for ontology-based source code interoperability [dAZS21].

The first phase (i.e. specification) defines the scope of the transformation. The subdomain is agentic AI systems, the source language is Python, and the target frameworks are CrewAI, LangGraph and AutoGen. The second phase, Subdomain Semantic Abstraction, is the construction of the domain ontology and is addressed by the ontology engineering step above. The third phase, Language Syntactic Abstraction, defines how source code is parsed into syntactic structures that the transformation can operate on. The fourth phase, Language Semantic Abstraction, maps these syntactic structures to the corresponding instances of the ontology.

The fifth phase, Application Perspective, specifies what is done with the populated ontology. The original OSCIN method applies this phase to analytical operations over the semantic graph such as querying, comparison and auditing [dAZS21]. This thesis

extends the fifth phase with code generation: the populated ontology is treated not only as an analytical artefact but also as a source from which framework-specific source code can be regenerated. This extension is what makes the transformation bidirectional and is one of the contributions of this thesis.

The transformation pipeline is deterministic. It operates through static parsing and mapping rules. Constructs that cannot be resolved without interpretation of natural-language content or semantic analysis of source code are identified but excluded from the pipeline, in order to preserve reproducibility. The resulting gaps in coverage are reported as part of the evaluation and discussed as limitations and directions for future work. The technical realization of the pipeline, including parser implementation, intermediate data structures and code generation, is presented in Chapter 6.

3.4 Evaluation

The evaluation is structured along two complementary dimensions, reflecting the inter-dependent nature of both research artifacts (i.e., AgentOscin and the corresponding transformation method).

Ontology Evaluation

The first evaluation dimension assesses the quality of AgentOscin as a semantic representation for agentic AI systems. The evaluation examines whether the ontology captures enogh of the architectural concepts required to support cross-framework representation, following the validation approach outlined in Ontology Development 101 [NM01] and extending it with a comparative baseline. AgentOscin is evaluated by checking, for each competency question defined during ontology engineering, whether the question can be answered from the structure the ontology provides.

To assess whether the proposed extensions add more expressiveness beyond existing representations, the same competency questions are evaluated against comparable data models from the literature analyzed in chapter 4. AgentO [EKEK26] serves as the primary baseline, as it is the only existing ontology designed specifically for agentic AI systems and the direct starting point for AgentOscin. The unified class model proposed by Derouiche et al. [DBM25] is included as a secondary comparison to cover the conceptual-model side of the literature. Coverage is classified on a three-level rubric: *full* when all concepts required to answer the question are representable in the model, *partial* when some but not all required concepts are representable, and *none* when the model provides no means of representing the concepts the question asks about. The resulting coverage profiles are compared across the models to quantify the expressiveness gain contributed by AgentOscin.

Transformation Experiments

The second evaluation dimension uses controlled transformation experiments to assess the operational usability of the ontology and the OSCIN-based transformation method.

In the *extraction direction*, framework-specific source code is transformed into instances of the semantic representation. The resulting instances are compared against the original source code to assess *transformation correctness* (the degree to which agentic AI concepts are accurately captured) and *transformation completeness* (the coverage of architectural elements such as agents, coordination structures, control flow and tool usage). Traceability is assessed by examining whether each semantic element can be linked back to a corresponding code-level construct through its mapping table entry.

In the *generation direction*, ontology instances are transformed into executable source code for a target framework. Correctness is evaluated by examining whether the generated code complies with the structural requirements of the target framework and reproduces the intended system architecture. Structural consistency is assessed by comparing the generated code against reference implementations where available.

To evaluate *cross-framework interoperability*, transformation experiments are conducted across all three frameworks. Source code from one framework is transformed into the semantic representation, regenerated into a second framework and then re-extracted into an ontology instance (see Figure 3.2). The original and reconstructed ontology instances are compared using structural equivalence criteria such as the preservation of agent roles, interaction patterns and control structures.

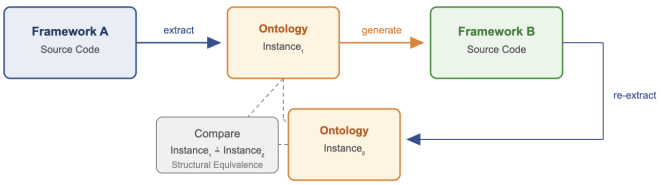

Figure 3.2: Cross-framework transformation experiment linking source code, semantic representation and regenerated target-framework code.

Across all experiments, the proposed approach is compared to an LLM-based baseline that uses large language models for direct code-to-model and model-to-code translation without formal semantic mappings. Comparison criteria include correctness, completeness, structural equivalence, consistency across transformation directions and reproducibility (whether the same input produces consistent outputs across repeated executions).

Together, the ontology evaluation and the transformation experiments provide the empirical basis for answering RQ4 by combining ontology-level quality assessment with evaluation of extraction and generation across heterogeneous frameworks.

CHAPTER 4

Agentic AI Concepts and Framework-level Implementation

The first work on Multi-Agent Systems and their architectures was conducted by [Woo09]. They defined core concepts like reasoning, agent interactions, coordination, and communication. Although their work was written before the current rise of generative AI, it still provides a solid foundation for understanding how modern agentic systems are designed. Recent studies have updated these classical ideas to fit LLM-based agents (e.g. [EKEK26, DBM25, Bot25]. This includes adding new elements like tool use, memory architectures and reasoning patterns.

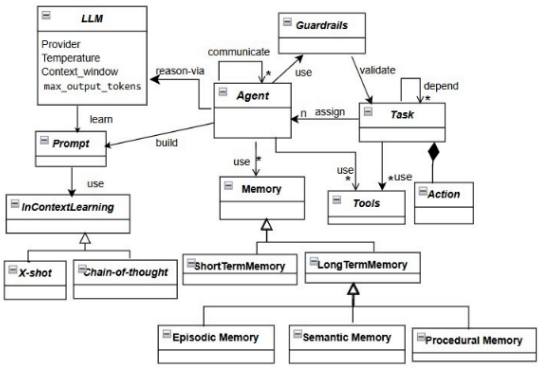

Figure 4.1: Unified class model showing the core components of agentic AI systems [DBM25]

[DBM25] created a unified model that shows the main parts of an agentic AI system (see Figure 4.1). Other researchers, such as [Bot25], suggest that the core of agentic AI systems can be simplified into three main pillars: Reasoning, Memory and Tool Use. Currently, there is no single, agreed-upon data model to categorize these designs in the

Table 4.1: Concept Matrix — Concepts for Agentic AI Frameworks

	Agent Definition			**Perception / Task**			**Coordination**			**Reasoning**			**Memory**			**Tool Use**		**Guardrails / Gov.**			
Paper	Agent Profile Schema	Role / Prompt Config.	LLM Backbone Sel.	Task Schema & Def.	Agent–Task Assign.	Deps. & Outputs	Orch. & Routing	Multi-Agent Topology	Termination & Flow	Planning & Decomp.	Reflection Mech.	Reasoning Pattern	Memory Scope	Short–Term Memory	Long–Term Mem. / RAG	Tool Schema & Reg.	Tool Assign. Scope	Rule & Constr. Def.	Intervention Mech.	HITL Interface	**Count**
[WJ95]	•	•					•	•		•											5
[Woo09]	•	•		•	•		•	•		•	•										8
[?]							•														1
[WBZ+23]	•	•		•	•		•	•	•				•	•		•	•			•	11
[HZC+24]	•	•		•	•	•	•	•		•						•	•	•			11
[LHI+23]	•	•					•	•													4
[YZY+23]										•		•				•					3
[GCW+24]	•	•		•	•	•	•	•		•	•	•	•			•					12
[LWZ+24]	•	•		•	•		•	•	•	•	•		•	•	•	•					13
[CLH+25]	•	•					•	•		•	•	•	•	•	•	•					11
[Sch25]	•	•	•				•	•		•	•	•	•	•	•	•					12
[NSSP26]	•						•	•		•	•	•	•			•					8
[YEL+25]										•		•				•	•	•			5
[BKN+25]	•	•		•			•	•		•		•	•	•	•	•		•			12
[Bot25]	•	•					•	•				•	•			•					7
[DBM25]	•	•	•	•	•		•	•				•	•	•	•	•		•			13
[WZ25]	•	•	•	•			•	•		•					•	•					9
[SRK26]	•	•					•	•	•			•	•		•	•					9
[AKA25]	•						•	•				•	•			•		•	•	•	9
[AAD25]																		•	•	•	3
[VRB26]				•	•	•	•	•	•							•					7
[PBPS25]						•										•	•				3
[MRV+25]																		•	•		2
[?]							•	•	•												3
[?]								•	•									•			3
[?]				•	•					•											3
[?]																		•	•		2
[EKEK26]	•	•	•	•	•		•	•	•				•			•					10
Total	16	14	4	10	9	4	17	17	6	9	5	9	10	5	6	16	4	6	3	3	**173**

research community. Because of this, we propose a literature-derived concept-matrix 4.1. In the following section these concepts are described and then further analyzed at the source code level across CrewAI, AutoGen and LangGrph. [Cre25] [Mic25] [Lan25d].

4.1 Agent Definition

[WJ95] provide one of the earliest comprehensive surveys of agent theories, architectures and languages, establishing core properties such as autonomy, reactivity, proactivity and social ability as defining characteristics of an agent. They define agents as a computing system situated in an environment that is capable of flexible, autonomous action to meet its design objectives.

In the context of modern LLM-based multi-agent systems, the agent remains a central architectural component, but its characterization has changed from simple reactive/proactive agents to fully self-aware agents, that exhibit meta-cognitve awareness [NSSP26].

In the context of the modern AI landscape, an agent is based on a foundation model (like a Large Language Model) and pursues complex, multi-step goals with limited direct supervision [Sch25]. Additionally, these agents are often defined with specific profiles, roles, backgrounds and constraints that specify how they act within the agentic AI system [LWZ+24] [GCW+24].

4.1.1 Framework Analysis

CrewAI

CrewAI defines an Agent as an autonomous unit capable of performing tasks, making decisions, using tools, interacting with other agents, maintaining memory of interactions and being able to delegate tasks when allowed. In the source code, agents are defined either using YAML configuration or directly in code.To define an agent's profile, CrewAI provides a list of agent attributes that can be specified when designing an agentic system. The only required parameters are role, goal and backstory [Cre25].

```yaml
data_analyst:
  role: Data Analyst
  goal: >
    Extract actionable insights from structured datasets
    and surface relevant patterns to the research team.
  backstory: >
    A seasoned analyst with expertise in statistical methods
    and business intelligence who prioritises clarity over
    complexity when communicating findings.
```

Listing 4.1: CrewAI agent definition (YAML configuration).

AutoGen

AutoGen provides a set of preset agent types that all inherit from the BaseChatAgent class. All agents share a common set of attributes that define their identity: name, description and system_message, which are configured at agent instantiation. The name serves as a unique identifier within a multi-agent system the description communicates the

agent's purpose to other agents during orchestration and the system_message provides the behavioral instructions passed to the underlying language model.

Among the preset agents, AssistantAgent serves as a general-purpose agent intended for prototyping and educational use. For more specialized requirements, AutoGen allows developers to define custom agents by subclassing BaseChatAgent, enabling the implementation of domain-specific behavior that does not fit the provided presets [Mic25].

LangGraph

LangGraph, while part of the LangChain ecosystem, follows a fundamentally different paradigm at the source code level than LangChain's agent abstractions. LangGraph explicitly distinguishes between *workflows* and *agents*: workflows follow predetermined code paths with a fixed execution order, whereas agents are driven by a language model that dynamically determines its own sequence of actions, typically through iterative tool use.

LangGraph does not provide its users with a mechanism to configure specific agents with roles, descriptions, or backstories. Instead, behavior is constructed using discrete steps called *nodes*, connected via *edges* that define transitions and conditional branching. An agent in LangGraph is therefore not defined through identity parameters but rather emerges as a node in which a language model autonomously decides whether to invoke tools or return a final response, typically implemented through a react-style loop [Lan25d].

```
1
2 def data_analyst(state: MessagesState):
3     system = SystemMessage(content=(
4         "You are a data analyst with expertise in statistical "
5         "methods and business intelligence. Prioritise clarity "
6         "over complexity when communicating findings."
7     ))
8     response = model.invoke([system] + state["messages"])
9     return {"messages": [response]}
```

Listing 4.2: LangGraph agent definition as a node function

4.1.2 Synthesis and Requirements

The framework analysis of agent definitions revealed structural inconsistencies on an architectural level across all three frameworks. While CrewAI and AutoGen support defining individual agents with distinct characteristics and roles through dedicated parameters, LangGraph follows a fundamentally different approach in which agents are not explicitly declared but emerge from graph nodes that contain language model calls with tool access.

Despite these differences, all three frameworks share the underlying concept of an autonomous unit that is associated with a language model and guided by natural

language instructions. CrewAI expresses this through role, goal and backstory; AutoGen through name, description and system_message and LangGraph implicitly through prompt content within node functions.

From this analysis, the following representational requirements for agent definitions can be derived:

R1 Framework-independent agent identity. The ontology must represent agent identity as a separable concept that can be populated either from explicit configuration attributes (as in CrewAI and AutoGen) or inferred from prompt-level content within computational nodes (as in LangGraph).

R2 Distinction between orchestration-relevant and model-relevant attributes. The ontology must distinguish between agent attributes that serve the orchestration layer (e.g., AutoGen's description used for speaker selection) and those that instruct the language model (e.g., AutoGen's system_message, CrewAI's backstory). A single agent may carry both types of directive simultaneously.

R3 Prompt decomposition into structured components The ontology must decompose natural language prompt content into structured components so that equivalent prompts can be reconstructed for different target frameworks from these components.

R4 Flexible attribute cardinality. The required and optional attributes differ across frameworks. CrewAI mandates role, goal and backstory; AutoGen requires only name; LangGraph has no required agent-level attributes at all. The ontology must define a minimal set of identity properties while accommodating framework-specific extensions without loss of information.

R5 Agent functional typing. The ontology must represent the functional role an agent plays within the coordination structure by distinguishing between general-purpose agents, manager/orchestrator agents, and agents that serve as proxies for human input (independently of how this role is expressed in framework-specific class hierarchies).

4.2 Perception Layer and Task Definition

The perception layer is responsible for ingesting external inputs from the environment and translating them into internal representations that the agent can reason about [BKN+25]. While early foundation models relied almost exclusively on text, modern agentic systems are increasingly equipped with multi-modal perception capabilities, processing inputs including textual descriptions, visual images, auditory signals and spatial coordinates [LWZ+24] [WZ25].

Critically, agentic systems do not rely on rigid, explicit commands. Tthey act upon abstract, high-level user goals. The perception layer translates these goals, alongside

multi-modal context, into structured internal representations that drive the agent's behavior [BKN+25]. It therefore constitutes the boundary at which human intent enters the agent system and takes on a computationally tractable form.

In the broader agentic AI literature, the perception layer describes how agents ingest and internalize external input at runtime. At the design-time abstraction level addressed by this thesis, the concern is how developers specify the tasks and actions an agent system should perform [DBM25]. The following analysis examines how the three frameworks define tasks, assign them to agents and express dependencies and output expectations.

4.2.1 Framework Analysis

CrewAI

CrewAI lets developers define a Task as a specific assignment completed by an agent. Similar to the Agent , tasks can be defined as YAML configuration or directly in the code.

On a source code level, a task is created with two required parameters: description, a natural language statement of what the task entails and expected_output, a description of what the completed task should produce. Tasks are optionally assigned to a specific agent via the agent parameter; if no agent is assigned, a hierarchical process can delegate tasks based on agent roles.

Tasks can depend on other tasks through the context attribute, which accepts a list of tasks whose outputs should be available as input. CrewAI also supports task-level tool assignment, allowing a task to override the agent's default tools, as well as output formatting through structured types such as output_pydantic and output_json [Cre25].

AutoGen

In AutoGen, there is no dedicated task class or configuration object. Instead, a task is expressed as a message passed to an agent or team at through the run or run_stream methods via the task parameter, which accepts a plain string. The task description, expected output and any constraints must all be encoded within this single string. There is no built-in mechanism for defining task dependencies, structured output formats, or guardrails at the task level; such logic must be implemented through agent orchestration or custom code [Mic25].

LangGraph

LangGraph does not have an explicit task concept. What other frameworks model as a task is instead encoded in the graph structure itself: a node's function body contains the instructions (typically as part of the prompt passed to the language model) and the graph's edges define the execution order and dependencies between steps. Task descriptions, expected outputs and constraints are embedded within node implementations rather than declared as separate, configurable objects [Lan25d].

4.2.2 Synthesis and Requirements

The framework analysis of task definitions revealed three distinct abstraction levels. CrewAI provides tasks as first-class, richly configurable objects with dedicated attributes for descriptions, expected outputs, agent assignment, dependencies, guardrails and structured output types. AutoGen reduces a task to a plain string passed at runtime, with no structural separation between the task description and its constraints. LangGraph has no explicit task concept; task logic is distributed across node functions and graph edges.

From this analysis, the following representational requirements can be derived:

R6 Task as a first-class ontological concept. The ontology must represent tasks as distinct entities with their own properties, even when the source framework does not model them explicitly. For AutoGen, this requires extracting task semantics from run method; for LangGraph, from node function bodies and associated prompts.

R7 Agent-task assignment. The ontology must represent the relationship between a task and the agent responsible for its execution. This includes explicit assignment (CrewAI's agent parameter), implicit assignment through orchestration (AutoGen's team routing) and structural assignment through graph topology (LangGraph's node-to-function binding).

R8 Data dependencies. The ontology must capture ordering and data dependencies between tasks. CrewAI models these explicitly through the context attribute, LangGraph encodes them as graph edges and AutoGen relies on sequential team execution or manual orchestration. The semantic representation must normalize these into a unified dependency model.

R9 Expected output. The ontology must represent the expected output of a task, both as a natural language description and, where available, as a structured schema definition.

4.3 Coordination Mechanism

When tasks exceed the capacity of a single agent, Agentic AI employs MAS, where specialized agents coordinate their efforts to solve complex problems [Bot25]. The architecture of this coordination typically falls into several structural categories: centralized orchestration, where a single supervisor agent decomposes tasks and routes them to sub-agents; hierarchical structures, where agents operate in a tree-like command flow; and decentralized or peer-to-peer networks, where agents negotiate and collaborate directly without a central authority [BKN+25] [LWZ+24].

Beyond structural organization, the mechanisms of coordination are defined by the interaction paradigms among agents, which can be cooperative, adversarial (debate), or

mixed [LWZ+24]. In cooperative teamwork, agents share information and divide labor to reach a common goal. Conversely, adversarial debate involves agents presenting and defending conflicting viewpoints to cross-validate reasoning, reduce hallucinations and refine solutions [GCW+24]. To efficiently manage the flow of information during these collaborative processes, systems often utilize a Shared Message Pool, a publish-subscribe mechanism allowing agents to extract only the information relevant to their specific roles without requiring direct point-to-point communication [HZC+24].

4.3.1 Framework Analysis

CrewAI

CrewAI implements coordination at two distinct levels: *Crews* for organizing agents around a shared set of tasks and *Flows* for orchestrating multiple crews and coding tasks into larger workflows.

A Crew is a team of agents assigned to a list of tasks, governed by a *process* that determines execution order. CrewAI provides two process types: sequential, where tasks are executed in the order they are defined with the output of one task serving as context for the next and hierarchical, where a manager agent dynamically delegates tasks to other agents based on their roles and capabilities. The process type is set at crew creation via the process parameter. In hierarchical mode, either a manager_llm or a custom manager_agent must be specified.

Flows operate at a higher abstraction level, connecting multiple crews, individual agents and arbitrary Python functions into event-driven workflows. A Flow is defined as a Python class inheriting from Flow, with methods decorated as entry points (@start), listeners (@listen), or routers (@router). Listeners are triggered when a specified method completes, enabling sequential chaining. Conditional logic is supported through or_ (trigger when any dependency completes), and_ (trigger when all dependencies complete) and router methods that return string labels to direct execution to different branches. Flows maintain their own state, which can be either unstructured (dictionary) or structured (Pydantic model) and is accessible across all methods in the flow [Cre25].

AutoGen

AutoGen implements coordination through *teams*, which are groups of agents that share a conversation context and take turns responding according to a defined interaction pattern. A team is instantiated by passing a list of agents and a termination condition that determines when execution stops.

AutoGen provides several team presets, each implementing a different speaker selection strategy:

- RoundRobinGroupChat: Agents take turns in a fixed round-robin order. Each agent broadcasts its response to all other agents, maintaining a shared context.

- SelectorGroupChat: A language model selects the next speaker after each message, enabling dynamic, model-driven delegation.
- MagenticOneGroupChat: A generalist multi-agent configuration designed for solving open-ended web and file-based tasks.
- Swarm: Agents use HandoffMessage to explicitly signal transitions to specific other agents, enabling developer-defined handoff patterns.

```
1
2  termination = (
3      TextMentionTermination("DONE")
4      | MaxMessageTermination(max_messages=6)
5  )
6
7  team = RoundRobinGroupChat(
8      participants=[researcher, critic],
9      termination_condition=termination,
10 )
```

Listing 4.3: AutoGen RoundRobinGroupChat with combined termination conditions.

Termination conditions control when a team stops executing. These are passed at team creation and include conditions such as TextMentionTermination (stops when a specific word appears), TextMessageTermination (stops when a specific agent produces a text message) and ExternalTermination (stops from outside the team). Conditions can be combined using bitwise operators [Mic25].

LangGraph

LangGraph encodes all coordination logic explicitly through a directed graph composed of nodes and edges. There is no dedicated team, crew, or orchestration abstraction. The graph itself is the coordination mechanism. Nodes represent discrete computational steps (agent calls, tool execution, routing logic) and edges define the transitions between them. This gives developers full control over the execution flow but requires all coordination patterns to be manually constructed.

On a source code level, a graph is built using the StateGraph class, which is parameterized with a typed state schema. Nodes are added via add_node and transitions are defined through add_edge for unconditional transitions or add_conditional_edges for branching based on the output of a node. Conditional edges accept a routing function that inspects the current state and returns the name of the next node to execute.

LangGraph provides prebuilt utilities that implement common coordination patterns. The create_react_agent function constructs a ReAct-style loop where an agent node and a tool node alternate until the agent produces a final response. For multi-agent coordination, developers manually compose multiple agent nodes within a single graph and define the routing logic between them [Lan25d].

4.3.2 Synthesis and Requirements

The framework analysis of coordination revealed three fundamentally different paradigms. Despite these differences, all three frameworks ultimately express the same fundamental coordination concerns: which agents participate, in what order they act and under what conditions execution branches or terminates. The difference is whether these concerns are declared through configuration (CrewAI's process type), selected from presets (AutoGen's team types), or constructed from primitives (LangGraph's nodes and edges).

From this analysis, the following representational requirements can be derived:

R10 Coordination pattern as a named concept. The ontology must represent coordination patterns (sequential, hierarchical, round-robin, swarm, selector-based) as identifiable concepts, regardless of whether they are selected from a preset (CrewAI, AutoGen) or manually constructed in a graph (LangGraph). This enables cross-framework comparison at the architectural level.

R11 Agent-to-coordination binding. The ontology must capture which agents participate in a coordination structure and how they are bound to it. Either as members of a crew with assigned tasks (CrewAI), as participants in a team (AutoGen), or as nodes within a graph (LangGraph).

R12 Termination conditions. All three frameworks define conditions under which execution stops, though the mechanisms differ: CrewAI relies on task completion and flow structure, AutoGen uses explicit termination condition objects and LangGraph terminates when a node routes to the END sentinel. The ontology must capture termination semantics independently of these framework-specific mechanisms.

R13 Multi-level coordination. CrewAI's separation of crews (inner coordination) and flows (outer orchestration) has no direct equivalent in AutoGen or LangGraph, where a single abstraction handles both levels. The ontology must be able to represent nested coordination structures so that a CrewAI flow containing multiple crews can be captured without flattening the hierarchy, while also representing the single-level coordination of the other frameworks.

R14 Workflow step structure with typed transitions. The ontology must represent the execution structure of a coordination pattern as a sequence of discrete steps with typed transitions. This includes distinguishing entry points, exit points, and intermediate steps; representing unconditional and conditional transitions with associated routing logic; and linking each step to its associated task where applicable.

4.4 Reasoning Engine

The reasoning engine acts as the cognitive core of the agent, transforming high-level goals into actionable steps through multi-step reasoning and planning [LWZ+24]. In

contrast to traditional models that provide immediate, single-step responses, agentic AI performs a step-by-step solution process that includes problem analysis, planning, solving, reflection and validation [Sch25].

To implement this robust cognition, systems employ structured reasoning frameworks such as Chain of Thought (CoT), Tree of Thoughts (ToT) and Graph of Thoughts (GoT) [LWZ+24]. These architectural frameworks allow agents to explore multiple partial reasoning chains concurrently, evaluate the viability of intermediate results and perform self-correction before committing to a final action [Sch25]. Furthermore, reasoning processes can be categorized into one-step decision-making, where a task is decomposed and executed in a single logical flow and multi-step reasoning, which requires iterative reasoning cycles that maintain long-term consistency with the overall objective [GCW+24].

4.4.1 Framework Analysis

CrewAI

CrewAI provides reasoning as an explicit, configurable feature at the agent level. On a source code level, this is achieved by setting a boolean flag reasoning in the agent definition. When set to True, the agent reflects on a given task, evaluates possible approaches and refines its plan before executing an action. This reflection cycle repeats until the agent converges on a satisfactory plan or a configurable max_reasoning_attempts threshold, defined as an integer in the agent definition, is reached. Reasoning in CrewAI is therefore a framework-managed capability that is toggled per agent independently of the underlying language model [Cre25].

AutoGen

In AutoGen, reasoning is not exposed as a dedicated framework feature but is integrated into its conversation-based architectural paradigm. Whether and how an agent reasons depends primarily on the capabilities of the underlying language model. If a reasoning-capable model such as o3-mini is selected as the model client, the model itself handles multi-step reasoning internally without additional framework configuration. Alternatively, reasoning can be achieved architecturally by introducing a dedicated planning agent that decomposes a task into sub-steps using a language model before delegating them to other agents. In this case, reasoning emerges from the orchestration of multiple agents rather than from a single agent's configuration [Mic25].

LangGraph

LangGraph does not provide a built-in reasoning mechanism at the agent or node level. Since LangGraph models agentic behavior as explicit graph structures, reasoning patterns must be manually constructed by the developer through the graph topology itself. A Chain of Thought pattern, for instance, can be implemented as a sequence of nodes where

each node performs one reasoning step and passes its output via the shared state to the next. More complex patterns such as reflection or self-correction require the developer to define conditional edges that loop back to previous nodes based on evaluation criteria. LangGraph therefore offers maximum flexibility in designing reasoning workflows but places the full architectural decision making on the developer, as no reasoning behavior is provided by default [Lan25d].

4.4.2 Synthesis and Requirements

The framework analysis of reasoning capabilities revealed three different abstraction levels. CrewAI treats reasoning as a framework-managed, declarative feature that is toggled per agent through a boolean flag and bounded by a configurable iteration limit. AutoGen delegates reasoning either to the underlying language model's native capabilities or to the architectural composition of multiple agents, where a dedicated planning agent orchestrates task decomposition. LangGraph provides no built-in reasoning mechanism and instead requires developers to manually encode reasoning patterns through the graph topology itself.

From this analysis, the following representational requirement for reasoning can be derived:

R15 Reasoning as capability and pattern. The ontology must represent reasoning on three dimensions: (1) whether an agent has reasoning enabled as a declarative capability, (2) which reasoning pattern is employed if identifiable, and (3) whether reasoning originates from the framework, the language model's native capabilities, or the system's graph topology.

4.5 Memory Architecture

Memory provides agents with temporal continuity and the foundational knowledge required to navigate complex environments [BKN+25]. The architecture is typically divided into distinct functional durations. Short-term memory (or working memory) maintains the immediate conversational or task context within the language model's context window constraints [DBM25] [Sch25]. Long-term memory, conversely, captures persistent data across sessions, allowing the agent to recall historical interactions, user preferences and learned knowledge over time [DBM25].

Beyond duration, long-term memory can be functionally categorized into semantic memory for storing facts and reasoning paths, procedural memory for recalling specific task flows and strategies and episodic memory for retaining detailed contextual snapshots of specific past events [DBM25]. To manage and access this amount of information, agents frequently utilize Retrieval-Augmented Generation (RAG) and external vector databases. These systems retrieve relevant text snippets based on semantic similarity and append them to the agent's prompts, thereby reducing hallucinations and supporting

dynamic learning [GCW+24] [Sch25]. Memory operations also involve active reflection, where agents summarize and distill past experiences to continually enhance their cognitive abilities and adaptability [LWZ+24].

4.5.1 Framework Analysis

CrewAI

CrewAI provides an integrated Memory class that supports short-term, long-term, entity and external memory types. This class uses a language model to analyze content at storage time, inferring scope, categories and importance. The memory system can be used as a standalone component, at the agent level, at the crew level and within flows.

At the crew level, all agents in a crew share the crew's memory unless an individual agent has its own. After each task, the crew automatically extracts facts from the task output and stores them. Before the next task, relevant context is recalled from memory and injected into the task prompt at runtime. At the agent level, agents can maintain private memory that is not shared with the rest of the crew. Standalone memory acts as a general-purpose knowledge base that is independent of any specific agent or crew [Cre25].

AutoGen

AutoGen provides a Memory protocol that defines a standard interface for storing and retrieving contextual information. The protocol specifies methods for adding entries, querying relevant content, updating an agent's model context and clearing the store. The simplest implementation is ListMemory, which maintains memory entries in chronological order and appends them to the model's context before each task execution.

On a source code level, memory is assigned to an agent by passing a list of memory instances during agent instantiation via the memory parameter. At runtime, the agent queries its memory stores before processing a task and the retrieved content is injected into the model context as a system message. Memory in AutoGen is therefore agent-scoped: each agent manages its own memory instances independently.

For more advanced use cases, AutoGen provides memory extensions such as ChromaDBVectorMemory and RedisMemory, which implement the same Memory protocol but use vector similarity search for retrieval rather than chronological ordering. Since all implementations conform to the same protocol, they are interchangeable at the source code level without modifying the agent's logic [Mic25].

LangGraph

LangGraph implements memory through two distinct mechanisms that map to different persistence scopes. Short-term memory is provided through *checkpointers*, which persist the graph's state across invocations within the same thread. On a source code level, a

checkpointer is instantiated and passed to the graph at compile time via the checkpointer parameter. Each invocation is associated with a thread_id and the checkpointer automatically saves and restores the full graph state between turns, enabling multi-turn conversations.

Long-term memory is implemented through a separate *store* abstraction, which persists data across threads and sessions. A store is similarly passed to the graph at compile time via the store parameter. Unlike the checkpointer, which operates transparently on the entire graph state, the store must be explicitly accessed within node functions to read and write memory entries. Entries are organized into namespaces, typically scoped by user or application context [Lan25d].

4.5.2 Synthesis and Requirements

CrewAI provides a unified memory system that operates at both crew and agent level, with automatic fact extraction and injection at runtime. AutoGen exposes memory through a protocol-based interface that is strictly agent-scoped, with no built-in shared memory across agents. LangGraph separates memory into two distinct mechanisms (i.e. checkpointers for short-term state persistence and stores for long-term data) both scoped to the graph as a whole and accessed explicitly within node functions.

From this analysis, the following representational requirements can be derived:

R16 Memory scope as an explicit property. The ontology must represent at which level memory is owned and accessible: agent-private, shared among a group, or global to the system.

R17 Separation of short-term and long-term persistence. The ontology must represent the lifecycle of memory, whether it persists within a single execution, across executions within a thread, or across all sessions independently of how a given framework packages its memory features.

4.6 Tool Integration

Tool integration significantly expands an agent's capabilities beyond its static, pre-trained knowledge by enabling it to interact directly with external systems and physical or digital environments [Sch25]. By dynamically invoking functionalities such as web browsers, calculators, code interpreters and specialized APIs, agents can retrieve real-time data, perform complex computations and execute software commands [LWZ+24] [Sch25]. The process of utilizing a tool involves several seamless sub-tasks: intent recognition to determine when a tool is needed, function selection to choose the most appropriate tool, parameter-value mapping to extract required arguments and the actual execution and response generation [YEL+25].

Advanced agentic systems are not just limited to using pre-existing tools; they also demonstrate the ability to autonomously select, combine and even create new tools. Because language models can generate executable code, agents can write new Python utility functions or scripts on the fly, verify them through unit tests and deploy them as custom tools to solve novel problems. This dynamic tool integration reduces computational errors, enhances the interpretability of the agent's actions and embeds operational rules in a clear, reliable manner [Sch25].

4.6.1 Framework Analysis

CrewAI

CrewAI defines a tool as a skill or function that agents can utilise to perform actions. Tools can be assigned at two levels: at the agent level, where they become available across all tasks the agent executes and at the task level, where they override the agent's default tool set for that specific task.

On a source code level, custom tools are created either by subclassing BaseTool or by using the @tool decorator on a function. When subclassing, the developer defines a name, a description, an input schema via args_schema (a Pydantic model) and implements the _run method containing the tool's logic. When using the decorator, the function's docstring serves as the tool description that the agent uses to decide when to invoke it. CrewAI also provides a large library of pre-built tools (e.g. SerperDevTool, FileReadTool, PDFSearchTool) that can be instantiated and assigned to agents or tasks without custom implementation [Cre25].

AutoGen

AutoGen enables tool use through the AssistantAgent, which accepts tools via the tools parameter. A tool is most commonly defined as a Python function, which the agent automatically converts into a FunctionTool at runtime. The tool schema — including name, description and parameter types — is automatically generated from the function's signature and docstring and provided to the language model during execution.

Beyond individual tools, AutoGen introduces the concept of a *Workbench*, which is a collection of tools that share state and resources. The primary example is McpWorkbench, which connects to a Model Context Protocol (MCP) server and exposes its tools to the agent. A workbench is passed to the agent via the workbench parameter and operates alongside any individually assigned tools [Mic25].

LangGraph

LangGraph implements tool use through LangChain's tool infrastructure. Tools are defined as Python functions using the @tool decorator, where the function's docstring serves as the description and type hints define the input schema. For more complex inputs, a Pydantic model or JSON schema can be specified via the args_schema parameter.

Tools are integrated into the graph through a prebuilt ToolNode, which handles tool execution, parallel tool calls and error handling. The ToolNode is added to the graph like any other node and connected to the model node via a conditional edge (tools_condition) that routes execution to the tool node only when the model generates a tool call [Lan25d].

4.6.2 Synthesis and Requirements

All three frameworks share the fundamental concept of a tool as a callable function with a name, description and typed input schema that is exposed to a language model. The definition mechanism is also convergent: all frameworks support defining tools as decorated Python functions with auto-generated schemas. The key differences lie in how tools are assigned, scoped and integrated into the execution flow.

From this analysis, the following representational requirements can be derived:

R18 Tool as a framework-independent concept. The ontology must represent tools as entities with a name, description and input schema, independent of the framework-specific definition mechanism (decorator, subclass, or function).

R19 Tool assignment scope. The ontology must capture at which level a tool is bound: to an individual agent (all frameworks), to a specific task (CrewAI), or to a graph node (LangGraph).

4.7 Guardrails and Governance Mechanisms

Because agentic AI operates with high autonomy in dynamic environments, robust guardrails are essential to ensure that systems act safely, predictably and in alignment with human values. Guardrails consist of embedded technical constraints. Including goal safety protocols, schema validators and fail-safe mechanisms—that monitor and validate agent outputs before execution, preventing harmful actions or deviations from assigned objectives. By incorporating node validation, retry logic and early-stage trust layers, developers can secure agent workflows and mitigate risks like malicious code execution or data privacy breaches [AKA25] [DBM25].

On a broader systemic level, governance frameworks dictate the degree of human oversight and accountability required for agentic operations. This is frequently implemented through Human-in-the-Loop (HITL) architectures, where a human must explicitly approve critical decisions, or Human-on-the-Loop (HOTL) designs, where humans monitor autonomous execution with the power to intervene via override protocols. Governance also heavily relies on Explainable AI (XAI) techniques and comprehensive audit trails to log reasoning steps and tool invocations, ensuring that the agent's decision-making process remains transparent, legally compliant and accountable when errors occur [AKA25].

4.7.1 Framework Analysis

CrewAI

CrewAI implements guardrails at the task level through the guardrail and guardrails parameters. Two types are supported: function-based guardrails, which are Python functions that receive a TaskOutput and return a tuple indicating success or failure with feedback and LLM-based guardrails, which are string descriptions that the agent's language model evaluates against the output. Multiple guardrails can be chained sequentially, with each receiving the output of the previous one. A configurable guardrail_max_retries parameter limits how many times an agent reattempts a task after guardrail failure [Cre25].

AutoGen

AutoGen does not provide a dedicated guardrail mechanism at the task or agent level. Output validation must be implemented by the developer, typically through a dedicated critic agent that evaluates outputs and provides feedback within the team's conversational flow. This is a common architectural pattern in AutoGen rather than a framework feature: a validator agent is included as a team participant and a TextMentionTermination condition (e.g. listening for "APPROVE") controls when execution stops.

Human-in-the-loop is supported through two complementary mechanisms. First, a UserProxyAgent can be included as a participant in a team, prompting the user for input during its turn via a configurable input function. This approach is synchronous and blocks the team's execution until the user responds. Second, the team can be configured to pause after a set number of agent turns using the max_turns parameter, or to stop when an agent issues a HandoffMessage targeting the user, detected by a HandoffTermination condition. In both cases, the team preserves its state after stopping and can be resumed with user feedback passed as the task parameter in the next call to run [Mic25].

LangGraph

LangGraph does not provide built-in guardrails but offers comprehensive mechanisms for human-in-the-loop through its interrupt function. Unlike static breakpoints (which pause before or after specific nodes at compile time), interrupts are dynamic: they can be placed anywhere within a node's logic and can be conditional based on application state. When interrupt is called, graph execution is suspended, the current state is saved via the checkpointer and a payload is surfaced to the caller. Execution resumes when the caller re-invokes the graph with a Command(resume=...) containing the human's response, which becomes the return value of the interrupt call inside the node.

This mechanism supports several human-in-the-loop patterns at the source code level: approval workflows where a node pauses for a binary approve/reject decision, review-and-edit patterns where a human can modify LLM outputs before they are committed to state and input validation loops where an interrupt repeatedly prompts the user until

valid input is received. Interrupts can also be placed inside tool functions, causing the tool to pause for approval whenever it is invoked.

Guardrail-like validation must be implemented as a dedicated node in the graph that inspects the state and routes execution to either the next step or back to the producing node via conditional edges [Lan25d].

4.7.2 Synthesis and Requirements

The frameworks diverge significantly in how they handle validation and human oversight. CrewAI provides guardrails as a first-class, configurable feature at the task level with built-in retry logic and supports human-in-the-loop through task-level flags and flow-level decorators. AutoGen has no dedicated guardrail mechanism; validation is achieved through critic agents participating in the conversational flow, while human-in-the-loop is supported through UserProxyAgent participation, max_turns pausing and HandoffTermination conditions. LangGraph requires guardrail logic to be encoded as validation nodes with conditional edges and provides a dynamic interrupt function that can be placed anywhere within node logic for human review, approval, or input validation.

From this analysis, the following representational requirements can be derived:

R20 Guardrails as a task-level property. The ontology must represent that a task or step has associated validation logic, regardless of whether it is implemented as a task parameter (CrewAI), a validator agent (AutoGen), or a conditional node (LangGraph). The representation captures that validation exists and what criteria it checks, without modeling the implementation mechanism itself.

R21 Human-in-the-loop as an explicit checkpoint. The ontology must represent points in the workflow where human review or input is required. This includes CrewAI's human_input flag and @human_feedback decorator, AutoGen's UserProxyAgent and HandoffTermination targeting the user and LangGraph's interrupt calls within nodes. The semantic representation captures the presence and position of human checkpoints so that the transformation method can reconstruct an equivalent mechanism in the target framework.

4.8 Consolidated Requirements

The preceding sections analyzed how each agentic AI concept is realized across CrewAI, AutoGen, and LangGraph at the source code level. From the synthesis of each concept area, a total of 21 representational requirements were derived. Each requirement specifies what the semantic representation must capture to enable framework-independent modeling of the corresponding concept. Table 4.2 consolidates these requirements across all seven concept areas. Together, they define the scope and acceptance criteria for the ontology engineering phase that follows.

Table 4.2: Representational Requirements for a Semantic Model of Agentic AI Systems

ID	Concept	Requirement	Traceability
R1	Agent Def.	**Framework-independent agent identity.** The ontology must represent agent identity as a separable concept that can be populated either from explicit configuration attributes (as in CrewAI and AutoGen) or inferred from prompt-level content within computational nodes (as in LangGraph).	[Woo09, Bot25, WZ25, EKEK26, Cre25, Mic25, Lan25d]
R2	Agent Def.	**Distinction between orchestration-relevant and model-relevant attributes.** The ontology must distinguish between agent attributes that serve the orchestration layer and those that instruct the language model. A single agent may carry both types of directive simultaneously.	[Sch25, DBM25, WBZ+23, Mic25]
R3	Agent Def.	**Prompt decomposition into structured components.** The ontology must decompose natural language prompt content into structured components so that equivalent prompts can be reconstructed for different target frameworks from these components.	[GCW+24, DBM25, EKEK26]
R4	Agent Def.	**Flexible attribute cardinality.** The ontology must define a minimal set of identity properties while accommodating framework-specific extensions without loss of information.	[DBM25, EKEK26, Cre25, Mic25, Lan25d]
R5	Agent Def.	**Agent functional typing.** The ontology must represent the functional role an agent plays within the coordination structure by distinguishing between general-purpose agents, manager/orchestrator agents and agents that serve as proxies for human input.	[Bot25, SRK26, WBZ+23, Cre25]
R6	Perception / Task	**Task as a first-class ontological concept.** The ontology must represent tasks as distinct entities with their own properties, even when the source framework does not model them explicitly.	[HZC+24, VRB26, BKN+25, Cre25, **?**]
R7	Perception / Task	**Agent-Task assignment.** The ontology must represent the relationship between a task and the agent responsible for its execution, including explicit assignment, implicit assignment through orchestration and structural assignment through graph topology.	[GCW+24, LWZ+24, VRB26, Cre25, Lan25d, **?**]

Continued on next page

ID	Concept	Requirement	Traceability
R8	Perception / Task	**Data dependencies.** The ontology must capture ordering and data dependencies between tasks and normalize these into a unified dependency model.	[GCW+24, HZC+24, Cre25, Lan25b]
R9	Perception / Task	**Expected outputs.** The ontology must represent the expected output of a task, both as a natural language description and, where available, as a structured schema definition.	[HZC+24, VRB26, PBPS25, Cre25]
R10	Coordination	**Coordination pattern as a named concept.** The ontology must represent coordination patterns (sequential, hierarchical, round-robin, swarm, selector-based) as identifiable concepts regardless of whether they are selected from a preset or manually constructed in a graph.	[GCW+24, VRB26, CLH+25, Cre25, Mic25, **?**]
R11	Coordination	**Agent-to-coordination binding.** The ontology must capture which agents participate in a coordination structure and how they are bound to it.	[LWZ+24, WBZ+23, Cre25, Mic25, Lan25b, **?**]
R12	Coordination	**Termination conditions.** The ontology must capture termination semantics independently of framework-specific mechanisms.	[WBZ+23, Mic25, Lan25c]
R13	Coordination	**Multi-level coordination.** The ontology must represent nested coordination structures so that a CrewAI flow containing multiple crews can be captured without flattening the hierarchy, while also representing single-level coordination.	[SRK26, LWZ+24, Cre25, **?**]
R14	Coordination	**Workflow step structure with typed transitions.** The ontology must represent the execution structure as a sequence of discrete steps with typed transitions, distinguishing entry points, exit points, intermediate steps, unconditional and conditional transitions with associated routing logic.	[VRB26, Lan25c, Lan25b, Cla26, **?**]
R15	Reasoning	**Reasoning as capability, pattern and origin.** The ontology must represent whether an agent has reasoning enabled, which reasoning pattern is employed if identifiable and whether reasoning originates from the framework, the model or the system topology.	[Sch25, YZY+23, NSSP26, YEL+25, Cre25]
R16	Memory	**Memory scope as an explicit property.** The ontology must represent at which level memory is owned and accessible: agent-private, shared among a group or global to the system.	[LWZ+24, DBM25, Cre25, Mic25]

Continued on next page

ID	Concept	Requirement	Traceability
R17	Memory	**Separation of short-term and long-term persistence.** The ontology must represent the lifecycle of memory independently of how a given framework packages its memory features.	[DBM25, SRK26, Sch25, Cre25, Lan25d]
R18	Tools	**Tool as a framework-independent concept.** The ontology must represent tools as entities with a name, description and input schema, independent of the framework-specific definition mechanism.	[YEL+25, Sch25, PBPS25, YZY+23, Cre25, Mic25]
R19	Tools	**Tool assignment scope.** The ontology must capture at which level a tool is bound: to an individual agent, to a specific task or to a graph node.	[HZC+24, Cre25, Mic25, Lan25a]
R20	Guardrails	**Guardrails as a task-level property.** The ontology must represent that a task has associated validation logic, regardless of whether it is implemented as a task parameter, a validator agent or a conditional node.	[AKA25, DBM25, MRV+25, Cre25, **?**]
R21	Guardrails	**Human-in-the-loop as an explicit checkpoint.** The ontology must represent points in the workflow where human review or input is required, capturing the presence and position of human checkpoints.	[AKA25, AAD25, WBZ+23, Cre25, Lan25d]

CHAPTER 5

A Framework-Agnostic Semantic Representation

The previous chapter analyzed how agentic AI concepts are realized across three frameworks and derived a set of representational requirements from those findings. These requirements describe what a framework-independent semantic representation must be able to express in order to support interoperability. This chapter presents the ontology developed to meet those requirements. The ontology is the first of the two artefacts of this thesis. Its purpose is specific: it serves as the common intermediate representation into which framework-specific source code is extracted and from which framework-specific source code is generated.

The ontology is developed following the Ontology Development 101 methodology proposed by [NM01]. The first step of this methodology is to determine the domain and scope of the ontology. The domain is agentic AI systems and the ability to represent cross-framework implementation details.

5.1 Scope and Competency Questions

The scope is defined through competency questions, as introduced by [GF95]. Competency questions are natural language queries that a fully populated ontology must be able to answer. They act as boundary conditions: if a populated ontology instance cannot answer a competency question, either the ontology is missing a required concept or the extraction has not captured it [NM01]. In this thesis, the competency questions are derived directly from the requirements established in Chapter 4. The corresponding competency question can be found in 7.2.

It is also important to be clear about what the ontology does not capture. Runtime state, execution history and dynamic behavior that only shows when a system runs are outside

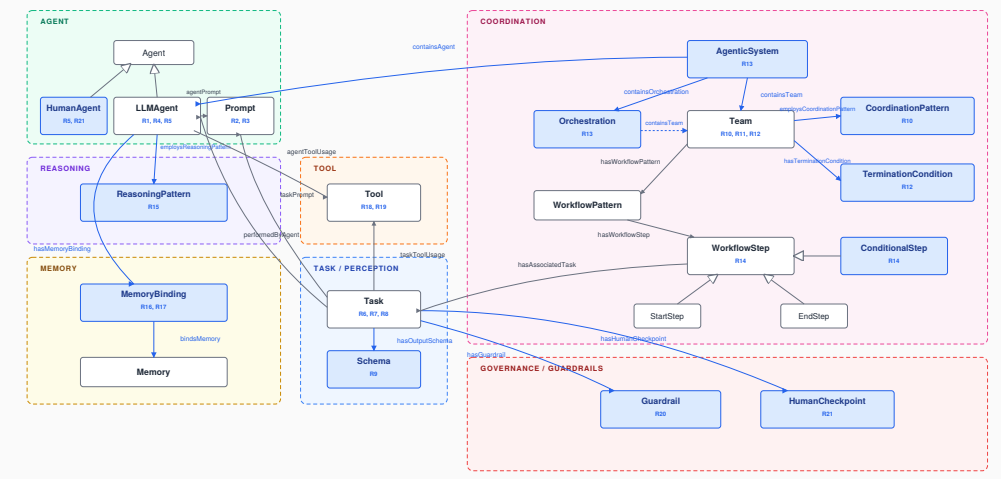

Figure 5.1: Excerpt: AgentOscin Concepts and Object Properties

scope. This includes conversation histories, dynamically selected speakers in AutoGen's SelectorGroupChat and the actual values passed through LangGraph's shared state.

5.2 Reuse Evaluation

The ontology is not built from scratch. AgentO [EKEK26] provides the conceptual foundation, defining core classes such as `LLMAgent`, `Task`, `Tool`, `Prompt` and `WorkflowStep` along with 22 object properties and 10 datatype properties. Where AgentO's existing vocabulary satisfies a requirement, it is reused without modification. Where it falls short, new classes and properties are introduced. All extensions are strictly additive: no existing AgentO class is removed or structurally modified and no existing property has its domain or range altered.

To determine where extensions are needed, we evaluated each requirement from Chapter 4 against AgentO's vocabulary. Table 7.1 summarises the results. Of the 21 requirements, AgentO fully covers 4 (R1, R4, R6, R11) through its existing agent identity, flexible cardinality, task and team membership classes. Eight requirements are partially covered: the relevant classes exist but lack properties needed for cross-framework transformation, such as distinguishing orchestration-facing from model-facing prompts (R2) or recording how task-agent binding was determined (R7). The remaining nine requirements address concepts entirely absent from AgentO, including coordination patterns, termination conditions, reasoning strategies, guardrails and human-in-the-loop checkpoints. The extensions derived from this analysis are detailed in the following section.

5.3 Defining Classes and Properties

This section presents AgentOscin, the ontology developed in this thesis. Figure 5.1 shows the resulting class structure and the principal object properties that connect them. Classes shown in gray are inherited from AgentO [EKEK26] without modification. Classes and properties shown in blue are introduced by AgentOscin to satisfy requirements that AgentO does not cover. Each extension is annotated with the requirement it addresses, providing a direct visual link between Chapter 4 and the ontological commitments made here.

The remainder of this section walks through the seven concept groups in the order established in Chapter 4. Rather than restating each class definition in prose, the text focuses on the design decisions that shaped the extensions and on the rationale for choosing one modeling approach over a plausible alternative.

5.3.1 Agent Definition

AgentO already provides the `LLMAgent` and `Prompt` classes, which together cover R1 (framework-independent agent identity) and R4 (flexible attribute cardinality) without modification. AgentOscin extends this baseline with three properties that carry the remaining agent-level requirements.

The property `agentType` (R5) types each agent by its functional role in the coordination structure rather than by its implementation class. The decision to model this as a property rather than as a subclass hierarchy reflects the cross-framework variability of the concept: a manager agent in CrewAI is structurally different from a `UserProxyAgent` in AutoGen, and both differ from a routing node in LangGraph [Cre25, Mic25, Lan25c]. A property-based encoding allows the same `LLMAgent` individual to be classified consistently regardless of how its host framework expresses the role.

R2 (orchestration- versus model-relevant directives) and R3 (prompt decomposition) are handled at the `Prompt` level rather than at the agent level. Distinguishing the directive function of a prompt fragment is an attribute of the prompt itself, not of the agent that carries it, and decomposing a prompt into structured components only makes sense once the prompt is treated as a first-class entity. AgentO's existing `Prompt` class is therefore retained as the attachment point for these properties.

5.3.2 Perception and Task Definition

The `Task` class is inherited from AgentO and satisfies R6 directly. AgentOscin adds four properties that capture task-level information not represented in AgentO.

`performedByAgent` (R7) records the agent responsible for a task. The associated property `hasDelegationStrategy` encodes how that assignment was determined: explicit assignment in the source configuration, structural assignment through graph topology, or delegation deferred to runtime. Recording the strategy alongside the

assignment preserves a distinction that is otherwise lost when the ontology is queried, since a CrewAI hierarchical process and a CrewAI sequential process produce structurally similar `performedByAgent` assertions for different reasons [Cre25].

`hasDependencyType` (R8) normalizes the three frameworks' distinct dependency mechanisms into a unified property. CrewAI's `context` attribute, LangGraph's edges, and AutoGen's implicit sequential ordering are heterogeneous on the surface but encode the same underlying relation between tasks [Cre25, Lan25c, Mic25]. The `hasDependencyType` value distinguishes these origins so that the generation direction can produce idiomatic code in each target.

`hasExpectedOutput` and `hasOutputSchema` (R9) separate the natural-language description of an expected output from its structured schema. The separation reflects the observation that all three frameworks support the former while only CrewAI consistently supports the latter, and that conflating them would force the ontology to either lose information or produce unreliable schemas during generation.

5.3.3 Coordination

The coordination concept group is the most extensively modified part of the ontology, reflecting the fact that AgentO does not represent coordination patterns or termination semantics as first-class concepts. Five new classes and several supporting properties are introduced.

`CoordinationPattern` (R10) reifies the coordination strategy as a named concept rather than encoding it as a string-valued property on `Team`. The reification is a deliberate choice. A property-valued encoding would prevent reasoning over the pattern itself, for instance asking whether two teams use compatible coordination strategies or whether a given pattern is supported by a target framework. With `CoordinationPattern` as a class, individual patterns (`Sequential`, `Hierarchical`, `RoundRobin`, `Selector`, `Swarm`) can be defined as instances and queried directly.

`TerminationCondition` (R12) follows the same modeling principle. AutoGen's termination conditions are first-class objects in the source language and combine using bitwise operators [Mic25]; representing them as ontology individuals preserves this compositional structure. The orange highlighting on `hasTerminationCondition` in Figure **??** marks a known asymmetry: termination conditions whose triggers depend on prompt content (such as `TextMentionTermination`) are structurally captured by the ontology but cannot be regenerated faithfully into frameworks where the prompt-trigger coupling is absent. This is recorded here and revisited in Chapter **??**.

The triple `AgenticSystem`, `Orchestration`, `Team` (R13) provides the multi-level coordination structure required to represent CrewAI flows containing multiple crews without flattening the hierarchy. `AgenticSystem` is the top-level container, `Orchestration` represents the outer flow and `Team` represents the inner coordination unit. A flat AutoGen team or a single LangGraph graph is represented as an `AgenticSystem` containing

exactly one `Team` and no `Orchestration`, ensuring that the same ontological structure accommodates both nested and single-level cases.

`WorkflowPattern` and `WorkflowStep` (R14) decompose the internal execution structure of a coordination unit. The `WorkflowStep` hierarchy distinguishes `StartStep`, `EndStep` and `ConditionalStep` as subclasses, attaching the routing logic property `hasRoutingLogic` at the `ConditionalStep` level. This avoids polluting unconditional steps with a property they cannot use and lets the generator emit unconditional or conditional transitions based on the step's class rather than on a runtime check.

R11 (agent-to-coordination binding) is satisfied by the existing `containsAgent` relation from `AgenticSystem` together with the `performedByAgent` relation on tasks. No new property is required.

5.3.4 Reasoning

R15 requires three distinct pieces of information to be representable: whether reasoning is enabled, what pattern it follows, and where the capability originates. AgentOscin models this with one new class and three properties on `LLMAgent`. The data property `hasReasoningEnabled` records the boolean enablement, `hasReasoningOrigin` records whether the capability comes from the framework, the underlying language model or the system topology, and `employsReasoningPattern` links to a `ReasoningPattern` individual when one can be identified.

Modeling reasoning origin as a property rather than encoding it implicitly in the agent's class is a deliberate choice driven by the extraction direction. CrewAI exposes reasoning as a configurable agent flag [Cre25], AutoGen leaves it to the underlying model [Mic25] and LangGraph requires it to be constructed in the graph topology [Lan25c]. These are observable distinctions in the source code but they do not warrant separate agent classes, since the agent itself remains structurally identical across the three cases.

5.3.5 Memory

The memory concept group introduces the `MemoryBinding` class to satisfy R16 (memory scope) and R17 (persistence separation). The binding is a reified relation between an `LLMAgent` and a `Memory` individual rather than a direct property. Reification is required because the same memory store can be bound at different scopes in different parts of a system. A CrewAI crew-level memory and a CrewAI agent-level memory may reference the same underlying store [Cre25] but differ in their access scope, and a direct property between agent and memory cannot distinguish these cases.

`hasMemoryScope` on `MemoryBinding` records whether the binding is agent-private, group-shared or system-global. `hasPersistenceScope` records whether the memory persists within a single execution, across executions on a thread, or across all sessions. Separating these dimensions preserves the lifecycle distinctions captured in LangGraph's

checkpointer-versus-store split [Lan25c] while allowing the same dimensional vocabulary to describe CrewAI's and AutoGen's memory configurations.

5.3.6 Tools

R18 (framework-independent tool concept) is largely covered by AgentO's existing `Tool` class. AgentOscin adds `hasInputSchema` to capture the typed input specification that all three frameworks generate from Python type hints, Pydantic models or JSON schemas. The property is a concrete extension rather than a class because the schema is structural metadata about the tool, not an entity in its own right.

R19 (tool assignment scope) is satisfied by introducing two distinct properties: `agentToolUsage` and `taskToolUsage`. Modeling the assignment as two separate properties rather than as a single property with a scope qualifier reflects the semantics of the target frameworks: agent-level and task-level tool assignment are not the same relation and conflating them would force the generator to infer scope at code emission time.

5.3.7 Guardrails and Governance

The governance concept group introduces two new classes to satisfy R20 and R21, both of which describe concepts entirely absent from AgentO.

`Guardrail` (R20) attaches to `Task` via the `hasGuardrail` property and carries `hasGuardrailType` to distinguish function-based, LLM-based and structural validation mechanisms. The class-level modeling is necessary because a task may have multiple guardrails chained sequentially [Cre25], which a flat property could not represent.

`HumanCheckpoint` (R21) attaches to `Task` via `hasHumanCheckpoint` and carries `hasCheckpointType` to distinguish the three observed checkpoint mechanisms: approval, review-and-edit and input-validation. Modeling human-in-the-loop as a positional checkpoint rather than as a boolean flag captures the information needed by R21, namely the position of the checkpoint in the workflow rather than merely its existence.

The auxiliary class `HumanAgent` extends the agent hierarchy to represent users participating in agentic systems through proxies such as AutoGen's `UserProxyAgent` [Mic25]. The `humanParticipatedIn` relation links a `HumanAgent` to the tasks or checkpoints it interacts with, enabling the ontology to distinguish agent-driven and human-driven contributions to a workflow without reclassifying users as ordinary agents.

5.3.8 Summary

AgentOscin extends AgentO with nine new classes and a corresponding set of object and data properties, all introduced as additive extensions that leave the AgentO baseline structurally intact. Every extension traces back to a requirement in Chapter 4 and is positioned in Figure 5.1 alongside its requirement tag. The next chapter shows how the resulting vocabulary is exercised by the bidirectional transformation method, both as

a target for extraction from framework-specific source code and as a source for code regeneration.

CHAPTER 6

Bidirectional Transformation Method

This chapter presents the transformation method that links framework-specific source code to the semantic representation developed in Chapter 5. The method is grounded in the OSCIN framework [dAZS21] for ontology-based source code interoperability, with two extensions motivated by the specific characteristics of the agentic AI domain. Section **??** maps the OSCIN phases to this thesis context and states the scoping decisions. Section 6.1.1 presents the extraction direction: three framework-specific extractors that transform Python source code into ontology instances. Section 6.1.2 presents the generation direction, which is based on a a template-based code synthesiser that transforms ontology instances into executable source code.

6.1 OSCIN Instantiation

The OSCIN method [dAZS21] defines five phases for achieving ontology-based source code interoperability: (1) Specification, which defines the transformation scope; (2) Subdomain Semantic Abstraction, which constructs the domain ontology; (3) Language Syntactic Abstraction, which defines parsers for each source language; (4) Language Semantic Abstraction, which maps syntactic structures to ontology instances; and (5) Application Perspective, which implements domain-specific operations over the populated ontology. Table 6.1 maps these phases to the thesis.

6.1.1 Extraction Method

In this phase, framework-specific source code patterns are mapped to a unified semantic representation. These mappings serve as the foundation for the automated transformation into the domain ontology.

Table 6.1: Mapping of OSCIN phases to the thesis context.

OSCIN Phase	**Thesis Chapter**	**Instantiation**
1. Specification	Ch. 3 (Methodology)	Subdomain: agentic AI systems. Language: Python. Frameworks: CrewAI, LangGraph, AutoGen v0.4. Perspective: cross-framework interoperability.
2. Subdomain Semantic Abstraction	Ch. 5	AgentOscin ontology: new classes, object properties, data properties extending AgentO.
3. Language Syntactic Abstraction	§6.1.1	Python ast module provides the parser. Framework-specific AST patterns are defined per extractor.
4. Language Semantic Abstraction	§6.1.1	Deterministic AST mapping for structural constructs
5. Application Perspective	§6.1.2	Code generation from ontology instances into CrewAI. **Extension of OSCIN** (generation direction).

The extraction process is structured into three distinct layers to ensure framework-agnostic extensibility:

1. **Framework-Specific Parsing (Layer 1):** Individual parsers use Python's `ast` (Abstract Syntax Tree) module to statically analyze source files.
2. **Shared Intermediate Representation (Layer 2):** The output of Layer 1 is mapped to a set of framework-neutral dataclasses (e.g., `ExtractedAgent`, `ExtractedTask`, `ExtractedFlow`). This Shared Intermediate Representation (SIR) acts as a structural contract, abstracting away framework-specific terminology.
3. **Ontology Population (Layer 3):** A framework-agnostic populator take the SIR objects as input to instantiate individuals within the *AgentOscin* ontology using `rdflib` [RDF25].

Framework-Specific Parsing via Static Analysis (Layer 1)

The first layer employs static program analysis to interpret the syntactic topology of the framework-specific source-code. By utilizing Python's built-in ast module, the parser constructs an Abstract Syntax Tree of the target source files [Pyt24]. The parsers implement the Visitor pattern (via ast.walk or ast.NodeVisitor) to traverse the tree and identify framework-specific syntactic structures. For instance, the AutoGenParser scans

for variable assignments where the value is an instantiation of AssistantAgent, extracting the arguments passed to the constructor. Conversely, the CrewAIParser detects classes decorated with @CrewBase and extracts functions decorated with @agent or @task, simultaneously resolving external YAML configuration files bound to the class attributes.

```
1
2           # Map to Shared Intermediate Representation
3           extracted_agents.append(ExtractedAgent(
4               role=agent_name,
5               backstory=sys_msg
6           ))
```

Listing 6.1: Simplified AST traversal for AutoGen agent extraction.

Shared Intermediate Representation (Layer 2)

To decouple the extraction logic from the ontology population logic, Layer 1 maps all framework-specific abstractions into a Shared Intermediate Representation (SIR). The SIR is implemented as a set of strictly typed Python dataclasses. This layer acts as an ontological proxy. This proxy normalizes framework implementations into a unified schema. For example, both an AutoGen AssistantAgent and a LangGraph add_node() invocation are reduced to an ExtractedAgent object, capturing only the semantic essence required by the ontology (e.g., role, tools, language model).

Ontology Population and Semantic Assertion (Layer 3)

The final layer iterates over the SIR objects to populate the *AgentOscin* ontology using the rdflib library. This process involves minting unique Uniform Resource Identifiers (URIs) for each extracted construct and asserting Resource Description Framework (RDF) triples.

The populator enforces the semantic rules defined in the TBox (schema). For example, when instantiating an agent, the populator creates an individual of type agentoscin:LLMAgent, generates an associated agentoscin:Prompt individual to store the backstory, and asserts an agentoscin:useLanguageModel object property linking the agent to its designated LLM.

```
7
8  # Assert Class Type
9  graph.add((agent_uri, RDF.type, AGENTOSCIN.LLMAgent))
10 graph.add((agent_uri, AGENTOSCIN.agentRole, Literal(agent.role)))
11
12 # Assert Prompt Object Property
13 if agent.backstory:
14     prompt_uri = EX[f"AgentPrompt_{sanitize(agent.name)}"]
15     graph.add((prompt_uri, RDF.type, AGENTOSCIN.Prompt))
16     graph.add((prompt_uri, AGENTOSCIN.promptContext, Literal(agent.
           backstory)))
```

```
17      graph.add((agent_uri, AGENTOSCIN.agentPrompt, prompt_uri))
```

Listing 6.2: Ontology population mapping SIR objects to OWL individuals using RDFLib.

6.1.2 Source Code Generation and Interoperability

In this thesis, we extend the scope of this final phase. Rather than strictly querying the ontology for analytical purposes, we utilize the semantic graph as a middleman to enable bidirectional interoperability. This is achieved through the reverse transformation: translating the populated, semantically enriched *AgentOscin* ontology back into executable source code for Agentic AI frameworks.

The generation pipeline operates in a reversed three-layer process:

1. **Ontology Reading (Layer 1):** A framework-agnostic reader utilizes `rdflib` to parse the serialized OWL ontology (e.g., Turtle format). It traverses the semantic ABox graph, identifying instances of core conceptual classes (e.g., `agentoscin:LLMAgent`, `agentoscin:WorkflowStep`) and resolving their associated object properties. This effectively acts as an advanced realization of OSCIN's Application Perspective Phase.

2. **Shared Intermediate Representation (Layer 2):** The queried semantic data is reconstructed into the identical Shared Intermediate Representation (SIR) data-classes used during extraction. By utilizing the SIR as a central pivot point, the architecture inherently supports an $N \times M$ translation matrix without requiring direct, tightly-coupled framework-to-framework translation logic.

3. **Framework-Specific Generation (Layer 3):** Dedicated generator modules (e.g., `AutoGenGenerator`, `LangGraphGenerator`) consume the populated SIR objects. Using templating engines (e.g., Jinja2) and abstract syntax manipulation, this layer synthesizes the syntactically correct boilerplate, configuration files, and orchestrational logic required by the target framework.

Ontology Reading and IR Reconstruction (Layers 1 & 2)

The reader component serves as the semantic-to-syntactic bridge. It executes graph queries to traverse the assertions, systematically resolving linked entities. For instance, when querying an `agentoscin:LLMAgent`, the reader must also traverse the `agentoscin:agentPrompt` object property to retrieve the role and backstory, and the `agentoscin:useLanguageModel` property to retrieve the LLM configuration.

```
1  for agent_uri in graph.subjects(RDF.type, AGENTOSCIN.LLMAgent):
2      # Query literal data properties
3      role = graph.value(agent_uri, AGENTOSCIN.agentRole)
4
5      # Traverse object properties for complex relationships
```

```
    prompt_uri = graph.value(agent_uri, AGENTOSCIN.agentPrompt)
    backstory = graph.value(prompt_uri, AGENTOSCIN.promptContext)
        if prompt_uri else None

    llm_uri = graph.value(agent_uri, AGENTOSCIN.useLanguageModel)
    llm_model = graph.value(llm_uri, AGENTOSCIN.hasTitle) if
        llm_uri else "gpt-4o"

    # Reconstruct the framework-neutral dataclass
    extracted_agents.append(ExtractedAgent(
        name=generate_variable_name(role),
        role=str(role),
        backstory=str(backstory) if backstory else None,
        llm=str(llm_model),
        tools=[] # Tools are resolved via separate graph traversal
    ))
```

Listing 6.3: Reconstructing the Shared Intermediate Representation from OWL individuals using RDFLib graph queries.

Framework-Specific Code Generation (Layer 3)

The framework-specific generators are responsible for rendering the SIR into source code. This process accounts for the architectural paradigms of the target framework. For instance, if the target is CrewAI, the generator produces both a Python execution script and the accompanying `agents.yaml` and `tasks.yaml` configuration files. If the target is AutoGen, the generator synthesizes a conversational AgentChat implementation, converting the generic agents into explicit `AssistantAgent` instantiations.

```
def generate_autogen_agents(agents: List[ExtractedAgent]) -> str:
    code_blocks = []
    for agent in agents:
        # Map SIR fields to AutoGen AssistantAgent syntax
        code_blocks.append(f"""
{agent.name} = AssistantAgent(
    name="{agent.name.capitalize()}",
    model_client=OpenAIChatCompletionClient(model="{agent.llm}"),
    system_message="{agent.backstory}"
)""")
    return "\n".join(code_blocks)
```

Listing 6.4: Generating AutoGen-specific agent instantiations from the neutral SIR.

CHAPTER 7

Evaluation

7.1 Evaluation Design

Table 7.2: Competency Questions derived from Representational Requirements

CQ	Req.	Concept	Competency Question
CQ-1	R1	Agent Def.	Which agents are defined in this system and what identifies each one?
CQ-2	R2	Agent Def.	Which of an agent's prompts guide the orchestration layer and which instruct the language model?
CQ-3	R3	Agent Def.	What are the instruction, context and output indicator components of a given agent's prompt?
CQ-4	R4	Agent Def.	Which agent properties are explicitly specified and which are left undefined across different framework representations?
CQ-5	R5	Agent Def.	Is a given agent a general-purpose worker, a manager or a proxy for human input?
CQ-6	R6	Perception	What tasks does this system define and what does each task instruct the assigned agent to do?
CQ-7	R7	Perception	Which agent is responsible for a given task and how was that assignment determined?
CQ-8	R8	Perception	Which tasks must complete before a given task can start and what is the nature of that dependency?
CQ-9	R9	Perception	What output is a task expected to produce and is a structured output schema defined for it?
CQ-10	R10	Coordination	What coordination pattern does a given team follow?
CQ-11	R11	Coordination	Which agents belong to a given team and how are they bound to its coordination structure?

Continued on next page

CQ	Req.	Concept	Competency Question
CQ-12	R12	Coordination	Under what conditions does a team's execution stop?
CQ-13	R13	Coordination	Does a team contain sub-teams and if so how are they nested?
CQ-14	R14	Coordination	What is the step-by-step execution structure of a team's workflow and where does it branch conditionally?
CQ-15	R15	Reasoning	Does an agent use reasoning, what pattern does it follow and where does that capability originate?
CQ-16	R16	Memory	Is an agent's memory private, shared with the team or accessible system-wide?
CQ-17	R17	Memory	Does a given memory persist only during execution, across threads or across all sessions?
CQ-18	R18	Tools	What tools are available in this system and what inputs does each one expect?
CQ-19	R19	Tools	Is a given tool bound at the agent level or overridden for a specific task?
CQ-20	R20	Guardrails	Does a task have validation guardrails and what type of check do they perform?
CQ-21	R21	Guardrails	At which points in the workflow must a human provide input or approval?

7.2 Extraction Evaluation

7.3 Generation Evaluation

7.4 Round-Trip and Cross-Framework Evaluation

7.5 Comparison with LLM-Based Baseline

7.6 Discussion of Results

Table 7.1: AgentO competency coverage against derived requirements.

Requirement	**Cov.**	**Key extension**
Agent Definition		
R1 Agent identity	•	—
R2 Directive distinction	○	`hasDirectiveFunction`
R3 Prompt decomposition	○	`hasSourceAttribute`
R4 Flexible cardinality	•	—
R5 Agent functional typing	○	`agentType`
Perception Layer		
R6 Task as first-class	•	—
R7 Task-agent assignment	○	`hasDelegationStrategy`
R8 Task dependencies	×	`dependsOn`
R9 Task output	○	`hasExpectedOutput`, `hasOutputSchema`
Coordination		
R10 Coordination patterns	×	`CoordinationPattern` hierarchy
R11 Agent-coordination binding	•	—
R12 Termination conditions	×	`TerminationCondition` hierarchy
R13 Multi-level coordination	×	`Orchestration`, `AgenticSystem`
R14 Workflow step structure	○	`ConditionalStep`, `hasRoutingLogic`
Reasoning		
R15 Reasoning capability	×	`ReasoningPattern` hierarchy
Memory		
R16 Memory scope	○	`MemoryBinding`, `hasMemoryScope`
R17 Persistence separation	○	`hasPersistenceScope`
Tools		
R18 Tool as concept	○	`hasInputSchema`
R19 Tool assignment scope	×	`taskToolUsage`
Guardrails & Governance		
R20 Guardrails	×	`Guardrail`, `hasGuardrail`
R21 Human-in-the-loop	×	`HumanCheckpoint`

• = fully covered (4) ○ = partially covered (8) × = not covered (9)

CHAPTER 8

Discussion

8.1 Interpretation of Findings

8.2 Threats to Validity

8.3 Relation to Research Questions

CHAPTER 9

Conclusion

9.1 Summary of Contributions

9.2 Limitations and Future Work

Overview of Generative AI Tools Used

List of Figures

3.1 Three cycle view of DSR according to [Hev07] 7
3.2 Cross-framework transformation experiment linking source code, semantic representation and regenerated target-framework code. 11

4.1 Unified class model showing the core components of agentic AI systems [DBM25] 13

5.1 Excerpt: AgentOscin Concepts and Object Properties 36

List of Tables

4.1 Concept Matrix — Concepts for Agentic AI Frameworks 14
4.2 Representational Requirements for a Semantic Model of Agentic AI Systems 31

6.1 Mapping of OSCIN phases to the thesis context. 44

7.2 Competency Questions derived from Representational Requirements . . . 49
7.1 AgentO competency coverage against derived requirements. 51

List of Algorithms

Bibliography

[AAD25] Hesham Allam, Ban AlOmar, and Juan M. Dempere. Agentic ai for it and beyond: A qualitative analysis of capabilities, challenges, and governance. *The Artificial Intelligence Business Review*, 2025.

[AKA25] Deepak Acharya, Karthigeyan Kuppan, and Divya B Ashwin. Agentic ai: Autonomous intelligence for complex goals – a comprehensive survey. *IEEE Access*, PP:1–1, 01 2025.

[BKN+25] Ajay Bandi, Bhavani Kongari, Roshini Naguru, Sahitya Pasnoor, and Sri Vidya Vilipala. The rise of agentic ai: A review of definitions, frameworks, architectures, applications, evaluation metrics, and challenges. *Future Internet*, 17(9), 2025.

[Bot25] V. Botti. Agentic ai and multiagentic: Are we reinventing the wheel?, 2025.

[Cla26] Bryan Clark. LangGraph, 4 2026.

[CLH+25] Shuaihang Chen, Yuanxing Liu, Wei Han, Weinan Zhang, and Ting Liu. A survey on llm-based multi-agent system: Recent advances and new frontiers in application, 2025.

[Cre25] CrewAI Inc. CrewAI documentation. `https://docs.crewai.com`, 2025. Accessed: April 2026.

[dAZS21] Camila Zacché de Aguiar, Félix Zanetti, and Vitor E. Silva Souza. Source code interoperability based on ontology. In *Proceedings of the XVII Brazilian Symposium on Information Systems*, SBSI '21, New York, NY, USA, 2021. Association for Computing Machinery.

[DBM25] Hana Derouiche, Zaki Brahmi, and Haithem Mazeni. Agentic ai frameworks: Architectures, protocols, and design challenges, 2025.

[EKEK26] Andreas Ekelhart, Kabul Kurniawan, Fajar J. Ekaputra, and Elmar Kiesling. Agento: An ontology for modeling agentic ai systems. In *The Semantic Web: 23th International Conference, ESWC 2026 (accepted for publication)*, 2026.

[EPK25] Fajar J. Ekaputra, Alexander Prock, and Elmar Kiesling. Towards supporting ai system engineering with an extended boxology notation. In *KG-STAR 2025 Knowledge Graphs for Responsible AI 2025*, volume 4018 of *CEUR Workshop Proceedings*. CEUR Workshop Proceedings, August 2025.

[FC18] Hector Florez and Marcelo Castro. Model driven engineering approach to configure software reusable components. In *Proceedings of the 1st International Conference on Applied Informatics (ICAI 2018)*, pages 352–363, November 2018.

[GB09] Maria Grant and Andrew Booth. A typology of reviews: An analysis of 14 review types and associated methologies. *Health information and libraries journal*, 26:91–108, 07 2009.

[GCW+24] Taicheng Guo, Xiuying Chen, Yaqi Wang, Ruidi Chang, Shichao Pei, Nitesh V. Chawla, Olaf Wiest, and Xiangliang Zhang. Large language model based multi-agents: A survey of progress and challenges, 2024.

[GF95] Michael Grüninger and Mark Fox. Methodology for the design and evaluation of ontologies. 07 1995.

[Hev07] Alan R. Hevner. A three cycle view of design science research. *Scandinavian Journal of Information Systems*, 19, January 2007.

[HMPR04] Alan R. Hevner, Salvatore T. March, Jinsoo Park, and Sudha Ram. Design science in information systems research. *Management Information Systems Quarterly*, 28:75–105, March 2004.

[HT19] Frank van Harmelen and Annette ten Teije. A boxology of design patterns forhybrid learningand reasoning systems. *Journal of Web Engineering*, 18(1):97–124, 2019.

[HZC+24] Sirui Hong, Mingchen Zhuge, Jiaqi Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. Metagpt: Meta programming for a multi-agent collaborative framework, 2024.

[Lan25a] LangChain, Inc. Langgraph api reference: `create_react_agent`, 2025. Accessed: 2026-03-01.

[Lan25b] LangChain, Inc. Langgraph api reference: `StateGraph`, 2025. Accessed: 2026-03-01.

[Lan25c] LangChain, Inc. Langgraph: Concepts — nodes, edges and state, 2025. Accessed: 2026-03-01.

[Lan25d] LangChain Inc. LangGraph documentation. https://langchain-ai.github.io/langgraph/, 2025. Accessed: April 2026.

[LHI+23] Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. Camel: Communicative agents for "mind" exploration of large language model society, 2023.

[LWZ+24] Xinyi Li, Sai Wang, Siqi Zeng, Yu Wu, and Yi Yang. A survey on llm-based multi-agent systems: workflow, infrastructure, and challenges. *Vicinagearth*, 1, 10 2024.

[Mic25] Microsoft Research. AutoGen documentation. https://microsoft.github.io/autogen/, 2025. Accessed: April 2026.

[MRV+25] Erik Miehling, Karthikeyan Natesan Ramamurthy, Kush R. Varshney, Matthew Riemer, Djallel Bouneffouf, John T. Richards, Amit Dhurandhar, Elizabeth M. Daly, Michael Hind, Prasanna Sattigeri, Dennis Wei, Ambrish Rawat, Jasmina Gajcin, and Werner Geyer. Agentic ai needs a systems theory, 2025.

[NM01] Natalya F. Noy and Deborah L. McGuinness. Ontology development 101: A guide to creating your first ontology. *Knowledge Systems Laboratory*, 32, January 2001.

[NSSP26] Ume Nisa, Muhammad Shirazi, Mohamed Ali Saip, and Muhammad Syafiq Mohd Pozi. Agentic ai: The age of reasoning—a review. *Journal of Automation and Intelligence*, 5(1):69–89, 2026.

[PBPS25] Tatiana Petrova, Boris Bliznioukov, Aleksandr Puzikov, and Radu State. From semantic web and mas to agentic ai: A unified narrative of the web of agents, 2025.

[Pyt24] Python Software Foundation. ast — Abstract Syntax Trees. https://docs.python.org/3/library/ast.html, 2024. Accessed: 2026-04-12.

[RDF25] RDFLib Team. RDFLib Documentation. https://rdflib.readthedocs.io/en/stable/, 2025. Accessed: 2026-04-13.

[Sch25] Johannes Schneider. Generative to agentic ai: Survey, conceptualization, and challenges, 2025.

[SRF18] Daniel Sanchez Rodriguez and Hector Florez. Model driven engineering approach to manage peripherals in mobile devices. In *Proceedings of the International Conference on Computational Science and Its Applications (ICCSA 2018)*, pages 353–364, July 2018.

[SRK26] Ranjan Sapkota, Konstantinos I. Roumeliotis, and Manoj Karkee. Ai agents vs. agentic ai: A conceptual taxonomy, applications and challenges. *Information Fusion*, 126:103599, February 2026.

[VRB26] Arunkumar V, Gangadharan G. R., and Rajkumar Buyya. Agentic artificial intelligence (ai): Architectures, taxonomies, and evaluation of large language model agents, 2026.

[WBZ+23] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W White, Doug Burger, and Chi Wang. Autogen: Enabling next-gen llm applications via multi-agent conversation, 2023.

[WJ95] Michael Wooldridge and Nicholas R. Jennings. Intelligent agents: theory and practice. *The Knowledge Engineering Review*, 10(2):115–152, 6 1995.

[Woo09] Michael Wooldridge. *An Introduction to MultiAgent Systems*. John Wiley Sons, 6 2009.

[WZ25] Christopher Wissuchek and Patrick Zschech. Exploring agentic artificial intelligence systems: Towards a typological framework, 2025.

[YEL+25] Asaf Yehudai, Lilach Eden, Alan Li, Guy Uziel, Yilun Zhao, Roy Bar-Haim, Arman Cohan, and Michal Shmueli-Scheuer. Survey on evaluation of llm-based agents, 2025.

[YZY+23] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models, 2023.